Automated IIITA Timetabling System

Group:-

Rahul Roy - IIT2023196
Lakvath Peer Singh - IIT2023197
Eshant - IIT2023198

Problem Statement

Core Problem: University timetabling is a complex matching problem classified as NP-hard.

- This means finding a perfect solution is too slow for large schedules, so we use heuristic algorithms instead.

Mathematical Objective: Minimize cost function **F(x)** where **H(x) = 0** (hard constraints) and **S(x) >= min** (soft constraints).

- Hard constraints are strict rules that must never be broken.
- Soft constraints improve schedule quality but are not strictly required.

Limitations of Current Approaches

Traditional Method: Manual spreadsheet-based planning is the traditional method used by academic departments to prepare college timetables.

Key Limitations:

a. **Human-Dependent:** Requires manual cross-verification of faculty and room availability.

b. **Difficult Tracking:** Difficult to enforce multiple constraints simultaneously (labs, electives, lecture distribution).

c. **Hidden Conflicts:** High probability of hidden conflicts in faculty, class, or room allocation.

d. **Restricted Optimization:** Limited ability to optimize schedules and reduce idle gaps.

e. **Slow Re-evaluation:** Timetable updates require manual re-evaluation of many schedules.

f. **Poor Scalability:** The process breaks down as the number of courses, faculty, and rooms increases.

g. **Core Gap:** Manual scheduling $\neq$ Conflict-free, optimized timetable generation.

Motivations

- **Primary Goal:** Build a conflict-free, automated timetable generator that produces high-quality schedules.
- **Reduce Manual Effort:** Eliminate the labor-intensive process of manually checking for professor and room overlaps.
- **Save Administrative Time:** Drastically reduce the weeks of time wasted by academic departments in preparing and revising timetables.
- **Human-in-the-Loop:** Provide a seamless web interface for administrators to make final manual adjustments directly on the generated grid.

Formulation of Hard Constraints

The solver evaluates fitness using a scalarized penalty function:

$$P(S) = \Sigma (W_{hard} * C_{hard}) + \Sigma (W_{soft} * C_{soft})$$

Hard Constraints (W_{hard}= 1000):

- **Time boundaries:** Sessions must not cross end-of-day or “break” boundaries.
- **Room Overlaps:** A physical room can host at most one session concurrently.
- **Professor Overlaps:** A professor can teach at most one session at a time.
- **Student Overlaps:** A group attends at most one session (except synced baskets).
- **Elective Sync:** Synced baskets must run concurrently; electives from different baskets must not be scheduled together

Formulation of Hard & Soft Constraints

Additional Hard Constraints (**W_{hard} = 1000**) handled by the algorithm:

- **WMC vs Section:** Whole-Batch classes strictly cannot overlap with section classes.
- **2+1 Lecture Spread:** Multi-period subjects must spread across different days.
- **Home Room & Labs:** Non-practical core sessions use assigned home rooms.

Soft Constraints (**W_{soft} = 1**):

- Minimizing empty slots between classes in the schedule.

Treating soft constraints at 1 ensures they only optimize when hard clashes hit zero.

Literature Review

Method / Technique	Key Idea	Advantages	Limitations	Reference
Genetic Algorithm (GA)	Represents timetables as chromosomes and evolves them using crossover and mutation.	Explores a large search space and improves solutions over generations.	Crossover frequently breaks structures, requiring complex repair tools.	Burke et al.
Multi-Objective Simulated Annealing (MOSA)	Uses simulated annealing to optimize multiple objectives simultaneously.	Can escape local optima and handle multiple objectives.	Slow convergence for datasets; robustness checks require expensive repairs.	Gülcü & Akkan
Evolution Strategy (1+1-ES)	Maintains a single solution and iterates using mutation only.	Lower cost (evaluates one candidate); avoids destructive crossover.	Requires extremely careful design of mutation operators.	Srivastava et al.
Improved ES with Preprocessing	Combines ES with preprocessing techniques and structured mutation.	Faster convergence speed and produces much higher-quality schedules.	Implementation complexity slightly higher.	Srivastava et al.

Limitation of GA and MOSA

Genetic Algorithm (GA) Flaws:

- **Requires Many Valid States:** GA needs a large initial population of valid timetables. In highly dense, constrained environments (like IIITA), generating many completely valid initial timetables randomly is nearly impossible.
- **Destructive Crossover:** Crossover operations break highly interdependent structures, such as synced elective baskets and lab sessions.

Multi-Objective Simulated Annealing (MOSA) Flaws:

- **The "Empty Slot" Trap:** In dense timetables (> 85% utilization), simple neighborhood moves fail because empty slots do not exist.
- **Constraint Destruction:** Random perturbations dismantle synchronized blocks, and balancing objectives with coin tosses prevents effective learning.
- **Cooling Schedule Mismatch:** Massive penalty weights for hard clashes, combined with misaligned cooling temperatures, freeze the algorithm in local optima.

The Shift to (1+1)-Evolution Strategy

The Pivot to (1+1) Evolution Strategy

- **Data Simplification:** Drastically simplified target representation by pre-chunking sessions upfront.
- **Memory Footprint:** (1+1)-ES holds only one parent and one offspring, making it memory light for browser execution, unlike population-based Genetic Algorithms (GAs).
- **Local Minima Management:** Implemented strict Rechenberg's 1/5th success rule to manage the local minima problem natively.
- **Adaptive Variance:** Scales search intensity without relying on brittle cooling schedules, allowing the solver to maintain constraint integrity through strict elitism.

System Design & Session Chunking

System Design (Three-Tier Architecture):

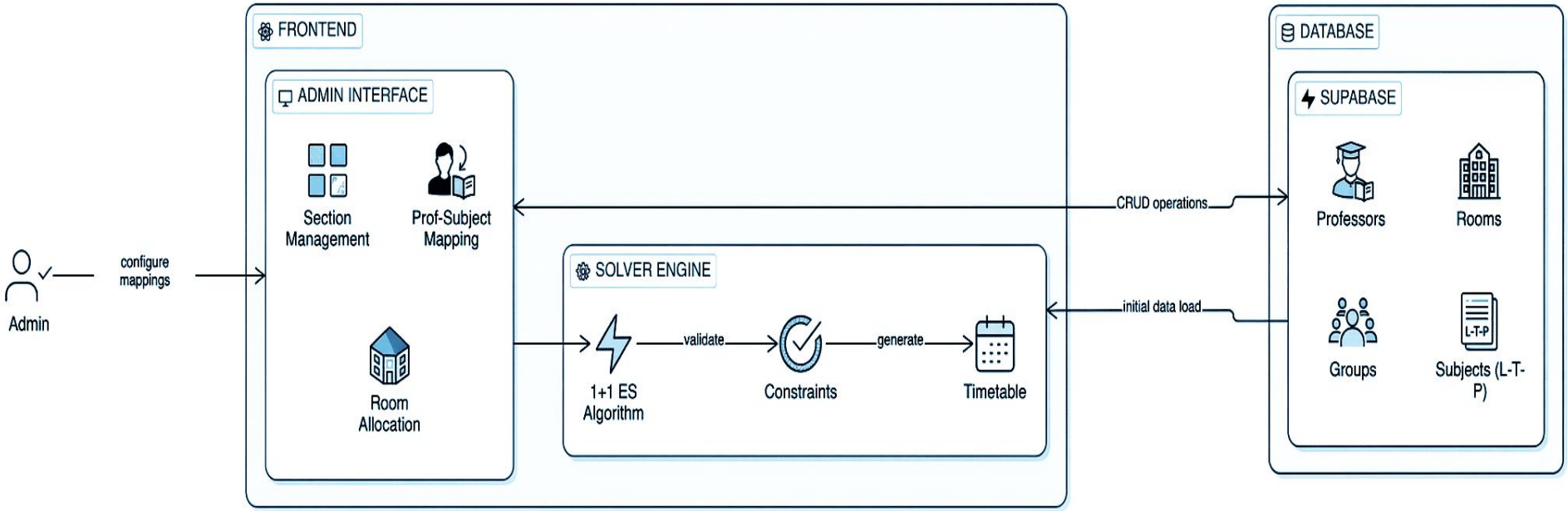
- **Database (Supabase):** Stores Professors, Rooms, Groups, and Subject L-T-P counts.
- **Frontend (React/TypeScript):** An interface for HODs to map professors to subjects and sections.
- **Solver (Client-Side):** An asynchronous engine running the optimization loop in the background of the browser.

Data Pre-processing (Session Chunking):

The system processes raw lecture hours (L) into chunks to reduce variable complexity:

- **L = 3:** One 2-hour contiguous block and one 1-hour block.
- **L = 2:** One 2-hour contiguous block.
- **Practicals (P):** One 2-hour contiguous block assigned to a Lab room domain.

System Workflow



The (1+1)-ES Algorithm (Part 1)

The engine follows an iterative loop to minimize a total penalty score P(S).

Step 1: Initialization. Initializes using a seed timetable ($\mathbf{S}_{\mathbf{parent}}$) and calculates its penalty.

Step 2: Mutation ($\boldsymbol{\sigma}$). A Child is created by selecting mutable sessions and applying five distinct operators governed by the variance sigma:

Mutators Used:

1. **Mutate Day:** Shifts the session to a random new day.
2. **Mutate Time:** Applies Gaussian noise scaled by σ.
3. **Mutate Room:** Reassigns strictly, respecting Lab vs. Lecture limits.
4. **Relocate (Global Jump):** Moves session to a totally new, random valid configuration (Day, Time, Room).
5. **Swap:** Trades places between two non-elective sessions of equal duration.

Sync Preservation: Any time mutation applied to an elective forces the entire synced basket to move with it.

The (1+1)-ES Algorithm (Part 2)

Step 3: Selection. The Child replaces the Parent only if $P(S_{parent}) <= P(S_{child})$. This elitism protects hard constraints.

Step 4: 1/5th Success Rule. Checked every **50** iterations:

If > 20% success: $\sigma = \sigma * 1.22$ (search wider).

If < 20% success: $\sigma = \sigma * 0.82$ (search finer).

Step 5: Stagnation Recovery.

- Checks for failure over 15,000 consecutive generations (a deep local optimum).
- It triggers a hard reset by generating a completely new initial timetable (preserving user-pinned UI classes).
- The tracking variance (sigma) is reset, but the global best timetable found so far is kept safe in memory against regression.

Conclusion

- **Final Result:** The implementation successfully manages the L-T-P division and the (1+1)-ES loop.
- **Overcoming Obstacles:** The system resolves hard conflicts in high-density scenarios where previous implementations failed.
- **Preserving Structure:** It specifically maintains the integrity of synchronized elective blocks through elitist selection.
- **Ready Structure:** The architecture serves as a functional, optimized solution for modern university timetabling needs.

Future Work

Human-in-the-Loop:

- Implementing and improving the React-based **Editor** functionality.

Quality Optimization:

- Addition of more complex Soft Constraints into the solver engine.
- **The current system reliably satisfies Hard Constraints, so future work will focus heavily on Soft Constraints (like minimizing professor campus time) to improve the human quality of life of the timetables.**